AI Image Library: API Contracts & Collaboration Workflow

This document provides a step-by-step guide on how our team will collaborate, what has been implemented on the frontend, and the exact REST APIs the backend developer needs to build to integrate with the frontend.

Part 1: Git Branching & Contribution Workflow

To keep the codebase stable and avoid conflicts, we use the Feature Branch Workflow. Do not commit directly to the `main` or `develop` branches.

Step-by-Step Command Flow for Teammates:

1. Clone the repository (For new setup)

```
git clone https://github.com/pushyanthdemoprojects/ai-image-library.git
cd ai-image-library
```

2. Synchronize branches

Always sync your local `develop` branch with the latest changes from GitHub before starting any work:

```
git checkout develop
git pull origin develop
```

3. Create your Feature Branch

Create a branch describing your task (e.g., `feature/backend-auth`, `feature/db-migrations`):

```
git checkout -b feature/YOUR_FEATURE_NAME
```

4. Commit changes locally

Use clear, descriptive commit messages:

```
git add .
git commit -m "feat: implement user registration and password hashing"
```

5. Push the feature branch to GitHub

```
git push -u origin feature/YOUR_FEATURE_NAME
```

6. Open a Pull Request (PR)

Go to GitHub, and click "Compare & pull request" next to your branch.

- **Target branch**: **`develop`** (do not select `main`).
- Request reviews from other team members. Once approved, merge it into `develop`.

Part 2: Frontend Implementation Overview

The frontend is built using React + Vite + Tailwind CSS (running on port `3000`). It has complete responsive layouts, form validation checks, and visual AI pipeline simulation states using mock datasets.

- **AuthContext & Hooks**: Local state storage that manages token persistence in the browser's `localStorage` and interceptors for Axios to append authorization tokens automatically.
- **Sign In / Sign Up (`/login` & `/register`)**: Beautiful glassmorphic forms featuring email formats checks and strength meters.

- **Drag-and-Drop Upload (`/upload`)**: Drag-and-drop container supporting local previews, 10MB filters, upload progress trackers, and detailed result sheets.
- **Gallery Grid & Search (`/search`)**: Search box supporting natural language queries, category filter chips (Nature, Tech, Animals, etc.), and hovering cards displaying captions and quick downloads.
- **AI Bounding Box details (`/image/:id`)**: Split metadata page rendering:
  - **YOLO Bounding Box Toggle**: Dynamically overlays absolute-positioned coordinates showing detected objects over the image.
  - **Dominant Colors**: Active color blocks with click-to-copy clipboard actions.
- **User Profile Dashboard (`/profile`)**: Storage limits progress bars, stats counters (uploads/downloads), and search query log listings.

Part 3: Backend API Contracts (For Member 2: Backend + AI)

The frontend makes requests to `http://localhost:8000` by default (or using the `VITE_API_URL` environment variable). The backend must implement the following endpoints:

1. User Registration

- **Endpoint**: `POST /register`
- **Authentication Required**: No
- **Request Body**:

```
{
  "username": "alex_developer",
  "email": "alex@example.com",
  "password": "StrongPassword123"
}
```

- **Response (HTTP 201 Created)**:

```
{
  "message": "User registered successfully",
  "user": {
    "id": 1,
    "username": "alex_developer",
    "email": "alex@example.com"
  }
}
```

- **Validations**:
  - `username`: Minimum 3 characters, alphanumeric or underscores.
  - `email`: Valid email format.
  - `password`: Minimum 8 characters, containing at least 1 letter and 1 number.

2. User Login

- **Endpoint**: `POST /login`
- **Authentication Required**: No
- **Request Body**:

```
{
  "email": "alex@example.com",
  "password": "StrongPassword123"
}
```

- **Response (HTTP 200 OK)**:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "user": {
    "id": 1,
    "username": "alex_developer",
    "email": "alex@example.com"
  }
}
```

3. File Upload & Processing

- **Endpoint**: `POST /upload`
- **Authentication Required**: **Yes** (`Authorization: Bearer <token>`)
- **Request Format**: `multipart/form-data`
- **Request Body**:
  - `file`: Binary file (acceptable types: JPEG, PNG, WEBP)
- **Response (HTTP 201 Created)**:

```
{
  "message": "Image uploaded and processed",
  "image": {
    "id": 101,
    "original_filename": "cat.jpg",
    "generated_filename": "ai_cat_playing_on_carpet_101.jpg",
    "caption": "A cute white kitten playing with a toy on a red carpet",
    "category": "Animals",
    "width": 1200,
    "height": 800,
    "file_size": 204857,
    "compressed_path": "compressed/ai_cat_playing_on_carpet_101.jpg",
    "thumbnail_path": "thumbnails/ai_cat_playing_on_carpet_101.jpg",
    "tags": ["cat", "kitten", "toy", "carpet", "playful"],
    "uploaded_at": "2026-07-29T21:00:00Z"
  }
}
```

4. Fetch All Images (Gallery)

- **Endpoint**: `GET /images`
- **Authentication Required**: No
- **Response (HTTP 200 OK)**:

```
[
  {
    "id": 1,
    "original_filename": "landscape.jpg",
    "caption": "Mountains reflecting on a calm lake",
    "category": "Nature",
    "image_path": "uploads/landscape.jpg",
    "thumbnail_path": "thumbnails/landscape.jpg",
    "tags": ["mountains", "lake", "nature"]
  }
]
```

5. Semantic Search

- **Endpoint**: `GET /search?q=<query>`
- **Authentication Required**: No
- **Response (HTTP 200 OK)**:
  - *Note: Runs CLIP semantic matching between the prompt string `<query>` and the vector database.*

```
[
  {
    "id": 1,
    "original_filename": "landscape.jpg",
    "caption": "Mountains reflecting on a calm lake",
    "category": "Nature",
    "image_path": "uploads/landscape.jpg",
    "thumbnail_path": "thumbnails/landscape.jpg",
    "tags": ["mountains", "lake", "nature"]
  }
]
```

6. Image Details (with YOLO and Colors)

- **Endpoint**: `GET /image/{id}`
- **Authentication Required**: **Yes** (`Authorization: Bearer <token>`)
- **Response (HTTP 200 OK)**:

```
{
  "id": 1,
  "original_filename": "golden_retriever_snow.jpg",
  "generated_filename": "ai_golden_retriever_playing_in_snow_1.jpg",
  "caption": "A happy golden retriever puppy playing in deep white snow during a sunny winter day",
  "category": "Animals",
  "width": 1920,
  "height": 1280,
  "file_size": 458291,
  "compressed_size": 145920,
  "image_path": "uploads/ai_golden_retriever_playing_in_snow_1.jpg",
  "tags": ["dog", "golden retriever", "snow", "puppy", "winter"],
  "uploaded_at": "2026-07-28T14:23:10Z",
  "detections": [
    { "label": "dog", "confidence": 0.96, "box": [15, 20, 65, 75] }
  ],
  "colors": [
    { "hex": "#FFFFFF", "name": "Snow White" },
    { "hex": "#B88E4C", "name": "Golden Fur" }
  ]
}
```

  - *Note: The `box` coordinates represent `[x, y, width, height]` as percentages (0 to 100) relative to the image container dimensions.*

7. Delete Image

- **Endpoint**: `DELETE /image/{id}`
- **Authentication Required**: **Yes**
- **Response (HTTP 200 OK)**:

```
{ "message": "Image and associated metadata deleted successfully" }
```

8. User Profile Dashboard Stats

- **Endpoint**: `GET /profile`
- **Authentication Required**: **Yes**
- **Response (HTTP 200 OK)**:

```
{
  "stats": {
    "total_uploads": 12,
    "total_downloads": 34,
    "storage_used": 12458290,
    "storage_limit": 52428800
  }
}
```

9. User Search History

- **Endpoint**: `GET /search-history`
- **Authentication Required**: **Yes**
- **Response (HTTP 200 OK)**:

```
{
  "history": [
          { "id": 1, "search_query": "golden retriever in snow", "searched_at": "2026-07-29T10:45:00Z" }
  ]
}
```

10. File Download Attachment

- **Endpoint**: `GET /download/{id}`
- **Authentication Required**: No
- **Response**: Binary download stream with `Content-Disposition: attachment; filename="<original_filename>"`